PRUEBA TÉCNICA · BANCO DE PREGUNTAS

Teoría: Java, Oracle y Angular

10 preguntas por tema con respuesta corta y una analogía con PHP / Laravel / Node / React. Las 1 a 5 son las básicas y de la 6 a la 10, alternativas esenciales. Al final, 5 extra de JWT y Spring.

Java 21 · Oracle · Angular 22 · JWT · Spring Boot · Quarkus

Proyecto: `login-jwt` (login con JWT, roles ADMIN y VIEW, CRUD sobre Oracle)

Preguntas de teoría: Java, Oracle y Angular

10 preguntas por tema: respuesta corta y una analogía (≈) con PHP, Laravel, Node o React. Las preguntas 1 a 5 son las básicas y de la 6 a la 10, las alternativas. Al final, 5 extra de JWT y Spring.

Java

J1. ¿Qué es la JVM? ¿JDK o JRE?
Java compila a *bytecode* y la JVM lo ejecuta en cualquier sistema operativo. El JRE sirve para ejecutar y el JDK para desarrollar (trae `javac`).
≈ La JVM es como el motor V8 de Node o el intérprete de PHP.

J2. Pilares de la POO. ¿Interfaz o clase abstracta?
Encapsulamiento, herencia, polimorfismo y abstracción. Una interfaz es un contrato y una clase puede implementar varias. Una clase abstracta puede tener estado y código, pero solo se hereda una.
≈ Igual que `interface` y `abstract class` en PHP.

J3. ¿`==` o `.equals()`?
Con objetos, `==` compara si es *el mismo objeto* en memoria y `equals()` compara el *contenido*. Los String siempre se comparan con `equals()`.
≈ Al revés de lo que uno espera en JS: en Java el `==` es el estricto.

J4. ¿Excepciones checked y unchecked?
A las checked (`IOException`, `SQLException`) el compilador obliga a manejarlas con `try/catch` o `throws`. Las unchecked (`RuntimeException`, `NullPointerException`) no. Para cerrar recursos: *try-with-resources*.
≈ PHP y JS solo tienen el equivalente a las unchecked.

J5. ¿List, Set o Map?
`List` tiene orden y permite duplicados. `Set` no permite duplicados. `Map` guarda pares clave-valor. Las implementaciones típicas son `ArrayList`, `HashSet` y `HashMap`.
≈ En PHP todo es `array`; en JS son `Array`, `Set` y `Map`.

J6. ¿Sobrecarga o sobrescritura?
La **sobrecarga** (*overload*) es el mismo nombre con distintos parámetros en la misma clase. La **sobrescritura** (*override*) es cuando una clase hija redefine un método del padre con `@Override`, y es la base del polimorfismo.
≈ PHP no tiene sobrecarga; la sobrescritura es igual.

J7. ¿Qué significan `static` y `final`?
`static` pertenece a la **clase**, no a cada objeto: se usa sin `new`. `final` significa que no cambia: una variable que no se reasigna, un método que no se sobrescribe o una clase que no se hereda. Juntos forman una constante: `static final`.
≈ `static` funciona igual que en PHP; `final` es como `const`.

J8. ¿Por qué el String es inmutable? ¿Qué es StringBuilder?
Cada "modificación" de un String crea uno nuevo, lo que lo hace seguro y permite reutilizarlo en memoria. Para concatenar muchas veces, por ejemplo dentro de un bucle, se usa `StringBuilder`, que modifica el mismo objeto.
≈ Como armar un texto con `.=` en un bucle de PHP, pero sin crear copias.

J9. ¿Qué son las lambdas y los Streams?

Una lambda es una función corta (`x -> x * 2`). Los Streams procesan colecciones de forma declarativa: `lista.stream().filter(...).map(...).toList()` .

≈ Son exactamente el `filter` y el `map` de los arrays de JS, o `collect()` en Laravel.

J10. Modificadores de acceso

`public` : desde cualquier lugar. `protected` : el mismo paquete y las subclases. Sin modificador (*default*): solo el mismo paquete. `private` : solo dentro de la clase. Los atributos van `private` , con getters y setters (encapsulamiento).

≈ Como en PHP, más el nivel "paquete", que PHP no tiene.

Oracle SQL

O1. ¿DDL, DML, DCL o TCL?

DDL define la estructura (`CREATE` , `ALTER` , `DROP`) y hace commit implícito. DML trabaja los datos (`SELECT` , `INSERT` , `UPDATE` , `DELETE`). DCL da permisos (`GRANT`). TCL controla la transacción (`COMMIT` , `ROLLBACK`).
≈ Igual que en MySQL, pero Oracle **no hace autocommit**.

O2. Tipos de constraints

`PRIMARY KEY` identifica la fila. `FOREIGN KEY` la relaciona con otra tabla. `UNIQUE` evita duplicados. `CHECK` valida una regla. `NOT NULL` obliga a tener valor. Se valida en la app para dar buenos mensajes y en la base para garantizar la integridad.
≈ Son las reglas de `validate()` de Laravel, pero dentro de la base.

O3. ¿INNER JOIN o LEFT JOIN?

INNER trae solo las filas que coinciden en las dos tablas. LEFT trae todo lo de la tabla izquierda y rellena con NULL lo que no tiene pareja.
≈ Invitados que confirmaron (INNER) frente a todos los invitados (LEFT).

O4. ¿WHERE o HAVING?

`WHERE` filtra **filas** antes de agrupar. `HAVING` filtra **grupos** después del `GROUP BY` , y por eso puede usar `SUM` o `COUNT` .
≈ WHERE elige a los alumnos; HAVING elige las secciones con promedio mayor a 8.

O5. ¿Qué es un índice?

Una estructura que encuentra filas sin recorrer toda la tabla. Acelera las lecturas en columnas de `WHERE` o `JOIN` , pero hace más lentas las escrituras. Para ver si se usa: `EXPLAIN PLAN` .
≈ El índice al final de un libro.

O6. ¿Qué es una transacción? ¿ACID?

Un grupo de operaciones que se confirman juntas (`COMMIT`) o se deshacen juntas (`ROLLBACK`). ACID: **A**tomicidad (todo o nada), **C**onsistencia, a**I**slamiento (las transacciones no se pisan entre sí) y **D**urabilidad (lo confirmado no se pierde).
≈ `DB::transaction()` en Laravel; en Java, `@Transactional` .

O7. ¿Procedimiento, función o trigger?

Un **procedimiento** ejecuta acciones y no devuelve valor (se llama con `EXEC`). Una **función** devuelve un valor y se puede usar dentro de un `SELECT` . Un **trigger** se ejecuta **solo**, ante un `INSERT` , `UPDATE` o `DELETE` (por ejemplo, para auditoría).
≈ Un trigger es como un *observer* o un evento de modelo en Eloquent, pero dentro de la base.

O8. ¿Qué es una vista (VIEW)?

Una consulta guardada con nombre que se usa como si fuera una tabla. No guarda datos (salvo las vistas *materializadas*). Sirve para simplificar consultas complejas y para mostrar solo ciertas columnas por seguridad.
≈ Un *scope* reutilizable de Eloquent.

O9. ¿Secuencia o IDENTITY?

Una **secuencia** es un objeto aparte que genera números (`seq.NEXTVAL`), el método clásico de Oracle. **IDENTITY** (desde 12c) es una columna que se autoincrementa sola. En JPA: `GenerationType.SEQUENCE` o `GenerationType.IDENTITY` .
≈ IDENTITY es el `AUTO_INCREMENT` de MySQL.

O10. ¿Cómo se maneja NULL?

NULL significa "sin valor": se compara con `IS NULL` o `IS NOT NULL`, **nunca** con `= NULL`. `NVL(col, 0)` reemplaza NULL por un valor. Además, `COUNT(col)` no cuenta los NULL y `COUNT(*)` sí.
≈ `NVL` es el `??` de PHP o JS.

Angular

A1. ¿Componente, servicio o standalone?

Un componente es una pieza de pantalla: TS, HTML y CSS. Un servicio es la lógica sin vista (la API o el estado compartido) y se inyecta. *Standalone* significa que cada componente declara sus propios `imports`, sin `NgModule`.

≈ Componente de React frente a un custom hook o contexto.

A2. Tipos de data binding

`{{ }}` y `[prop]` llevan datos de la clase a la vista. `(evento)` lleva eventos de la vista a la clase. `formControlName` o `[(ngModel)]` hacen las dos cosas. Control de flujo: `@if`, `@for` y `@switch`.

≈ `(click)` es el `onClick` de React.

A3. ¿Signals u Observables?

Un signal es un valor actual y síncrono (`cargando()`, `.set()`, `computed()`) y repinta la vista. Un Observable es un flujo asíncrono al que te suscribes (HttpClient, RxJS).

≈ Signal es `useState`; Observable es un *event emitter*.

A4. ¿Interceptor o guard?

El interceptor modifica **todas las peticiones HTTP**, por ejemplo para agregar el token. El guard decide si se puede **entrar a una ruta**, por ejemplo sin login → `/login`.

≈ El interceptor es un middleware de las peticiones que salen; el guard es el middleware `auth` de una ruta.

A5. ¿Formularios reactivos o de plantilla?

En los reactivos, el formulario y las validaciones van en el TS: tipados y fáciles de probar. Los de plantilla usan `ngModel` en el HTML y sirven para casos simples. Siempre se valida también en el backend.

≈ Reactivos son como `react-hook-form` con un esquema.

A6. ¿Constructor o `ngOnInit`?

El constructor (o `inject()`) solo recibe dependencias. `ngOnInit` se ejecuta cuando el componente ya está listo, y es donde se cargan datos. Otros hooks del ciclo de vida: `ngOnChanges` y `ngOnDestroy` (para limpiar suscripciones o *timers*).

≈ `ngOnInit` es el `useEffect(() => {}, [])` de React.

A7. ¿Cómo se comunican padre e hijo?

Del padre al hijo, con `input()` (antes `@Input`): `<hijo [dato]="x">`. Del hijo al padre, con `output()` (antes `@Output`), que emite un evento: `<hijo (guardado)="recargar()">`. Entre componentes que no están relacionados, con un servicio compartido.

≈ Las *props* y los *callbacks* de React.

A8. ¿Qué es un pipe?

Transforma un valor **para mostrarlo**, sin cambiar el dato: `{{ fecha | date:'dd/MM/yyyy' }}`, `{{ monto | currency }}`, `{{ texto | uppercase }}`. También se pueden crear pipes propios.

≈ Los filtros de Blade o Twig (`{{ nombre|upper }}`).

A9. ¿Cómo funciona el routing?

`app.routes.ts` asocia URLs con componentes, y `<router-outlet>` es el hueco donde se pintan. Admite parámetros (`/contribuyentes/:id`, que se leen con `ActivatedRoute`), *guards* y *lazy loading* (`loadComponent`), que carga el código solo cuando se visita la ruta.

≈ `routes/web.php` de Laravel, más el `@yield` del layout.

A10. ¿Qué aporta TypeScript?

JavaScript con **tipos**: los errores se detectan al compilar, hay autocompletado y existen `interface` para la forma de los datos (como `Contribuyente`). Al final se compila a JavaScript normal.

≈ JS con las *type hints* de PHP 8, pero revisadas antes de ejecutar.

Extra: JWT y Spring

X1. ¿Qué es un JWT?

Un token firmado con tres partes: `header.payload.firma`. El payload lleva el usuario, el rol y el vencimiento. **No está cifrado**, solo firmado: nadie lo puede alterar, pero cualquiera lo puede leer. El servidor no guarda sesión (*stateless*).

≈ Una pulsera de hotel con sello.

X2. ¿401 o 403?

401 significa "no sé quién eres": no hay token o es inválido. 403 significa "sé quién eres, pero no tienes permiso": por ejemplo, el rol VIEW intentando crear.

X3. ¿Por qué BCrypt?

Es un hash lento a propósito y con sal, así que la misma contraseña da hashes distintos y la fuerza bruta no es viable. Nunca se descifra: se compara con `matches()`.

≈ El `Hash::make()` de Laravel o `password_hash()` de PHP.

X4. ¿Qué es la inyección de dependencias?

La clase declara lo que necesita y el framework se lo entrega, en lugar de crearlo con `new`. Así queda desacoplada y es fácil de probar. Spring lo hace con `@Service` y el constructor; Quarkus, con `@ApplicationScoped` y `@Inject`.

≈ El *Service Container* de Laravel.

X5. ¿Spring Boot o Quarkus?

Hacen lo mismo: APIs REST, JPA y seguridad. Spring tiene más ecosistema y es más común en empresas. Quarkus hace el trabajo al compilar, así que arranca más rápido, gasta menos memoria y puede compilarse a binario nativo; está pensado para la nube.

≈ Laravel (completo) frente a un framework ligero para microservicios.